

DCLab R&D — Independent Review, Field Guide, and Roadmap

A plain-language walkthrough of `Research_Development_DCLab` for readers who were not part of building it — what it does, what it has already proven, and what to do next to turn it into a fine-tuning corpus for a small language model (SLM).

1. What this project actually is (in plain terms)

Most people who try to teach an AI “how to build a good ML model” hand it either (a) a few Kaggle notebooks, or (b) a textbook. Neither captures the messy, decision-heavy middle: *why* a feature is dangerous, *how* you’d notice a model is lying to you, *when* more feature engineering hurts instead of helps.

DCLab R&D is an attempt to manufacture that missing middle on purpose, as data:

- It picks **real, public, imperfect datasets** (currently 11, from the UCI repository: Adult, Bank Marketing, Breast Cancer, Credit Default, German Credit, Heart Disease, Mushroom, Online Shoppers, Spambase, Wine Quality — plus the proprietary HyperAck order-acceptance dataset).
- For each one, it runs the **same five-stage scientific procedure**, every time, on locked, honest train/validation/holdout splits.
- It writes down not just the final score, but the *reasoning trail*: what was checked, what looked suspicious, what was rejected and why, what is still uncertain.
- It uses an LLM as a **critic and hypothesis generator**, never as the thing that touches the data or computes the metric — a deliberate and important safety design (`AGENT_CONTEXT.md` , `DCLAB-R20`).

This isn’t a plan — it has already run. The `model_building_50_v1` campaign shows **50/50 experiments completed** (10 datasets × 5 stages), with a full holdout evaluation, calibration, and reliability check on every one of them. On top of that sits an older, larger registry: **579 historical experiment records across 11 datasets and 7 experiment suites**, and a live “Agentic Research Studio” that lets an LLM-driven agent propose and run new bounded experiments through a LangGraph loop.

Bottom line: the “50 experiments, from first EDA step to final trained model, with an LLM helping throughout” goal is not a future plan for this repo — it is already built and already executed once. What’s genuinely still open is turning that evidence into (a) a training corpus an SLM can actually be fine-tuned on, and (b) a second, more diverse round of experiments that stresses different failure modes than the first round did. Both are addressed in Sections 5–7.

2. The methodology, explained intuitively

Every experiment follows the same eight-step spine (`knowledge/workflow_blocks.json` , `MODEL_BUILDING_WORKFLOW.md`):

```
prediction contract
  ← immutable source + provenance
  ← locked holdout
  ← train-only EDA and leakage review
  ← fold-fitted feature ablation
  ← stability-aware algorithm screen
  ← conservative optimization
  ← one final holdout evaluation
  ← calibration / uncertainty / feature reliability
  ← evidence claims + LLM critic queue
```

Why this order matters, in everyday terms:

- **Prediction contract first.** Before looking at any data, write one sentence: *for [X], at [this exact moment], predict [Y], so we can [do Z]*. This single sentence is what actually decides whether a column is “cheating” or not — the same column can be perfectly fine for one prediction moment and pure leakage for another. Most feature-leakage arguments are really disagreements about what this sentence should say.
- **Lock the holdout before touching features.** If you peek at the test set while engineering features, your test set stops being a test set — it quietly becomes part of training. The rule here is blunt: consume the final holdout **once**, after every other decision is frozen (`DCLAB-R17`).
- **Everything gets fit only on the training fold** — imputers, encoders, scalers, feature selectors, calibration (`DCLAB-R03`). This sounds pedantic until you see the numbers below.
- **Screen several model families on identical folds before tuning any of them** (`DCLAB-R13`). Tuning one model in isolation and declaring victory is one of the most common ways teams fool themselves.
- **Optimize last, and only what already earned the right to be optimized** (`DCLAB-R15`). Hyperparameter search on a leaky or badly-engineered dataset just tunes the leakage harder.

3. What the evidence actually shows

3.1 Leakage: the single biggest score-inflator in the registry

Dataset	Honest (safe) ROC-AUC	Leaky (unsafe) ROC-AUC	Apparent, fake “improvement”
bank_marketing	0.803	0.936	+0.133
online_shoppers	0.774	0.931	+0.158
hyperack	0.945	0.980	+0.035

The intuition: `duration` in `bank_marketing` (the length of the sales call) and `PageValues` in `online_shoppers` are only known *after* the outcome already happened — a long call usually means the customer said yes; `PageValues` is Google Analytics' own estimate of purchase likelihood. HyperAck's `final_customer_fare` is only known after the trip is priced, i.e. after the order was already accepted. In every case, leakage bought **more apparent accuracy than any amount of legitimate tuning did** — a pattern that shows up repeatedly enough to be one of the project's clearest lessons.

The repo's leakage taxonomy is more useful than a simple "leaky / not leaky" flag — it separates **direct target leakage, post-outcome leakage, future leakage, window leakage, entity leakage (same person in train and test), preprocessing leakage, selection leakage** (re-using the test set for repeated decisions), and **external-data leakage** (a join that uses today's data instead of the data actually available at the historical prediction moment). A feature is judged leaky using four tests together — is it available at the exact prediction moment, what does its lineage/timestamp show, is it isolated from the split, and does removing/time-shifting it change the score materially — not from a suspicious column name or a high score alone (`DCLAB-R05` , `DCLAB-R06`).

3.2 No single “best” algorithm — but there is a *default*

Across all 11 datasets, no model family wins everywhere. LightGBM has the best **mean rank** (2.55, across 11 datasets), Extra Trees was the training-CV pick on 7 of the 10 controlled-campaign datasets, an engineered logistic regression led on churn, and a calibrated soft-voting ensemble (Extra Trees + LightGBM + XGBoost) is the HyperAck champion. The practical rule that falls out of this: **use gradient-boosted trees / Extra Trees as your default screen, but always screen at least a linear baseline and 2–3 nonlinear families side-by-side before committing** (`DCLAB-R13` , `DCLAB-R14`).

3.3 Feature engineering is not automatically good

Raw features (i.e., no engineering) won on **7 of 10** controlled-campaign datasets. Ratios helped on 2, interactions helped on 1. In the older HyperAck registry, expanding from 25 hand-picked features to 46 broadly-interactive ones actually *hurt* the gradient-boosted models ($\sim 0.945 \rightarrow \sim 0.937$) — a textbook case of feature dilution. The rule: **start from a strong raw baseline, and only promote a new feature if it clears a predeclared, meaningful margin over that baseline on identical folds** (`DCLAB-R07` , `DCLAB-R16`) — more features is not the same as more signal.

3.4 “Important” ≠ “usable in production”

This is the part of the ask about “which features are reliable / which ones will be missing in production” — the repo has a concrete answer (`DCLAB-R10`): feature importance is only a *nomination*. A feature is only production-reliable once it also passes:

1. **decision-time availability** — could you actually have this value at the moment you need to predict?
2. **stability across validation folds** — does its effect hold up, or is it noise from one lucky split?
3. **reproducible generation** — can you compute it the same way every time?

4. **contractual/operational availability** — will the upstream system that produces it still exist and be reachable in production?
5. **missingness and drift monitoring** — what happens when it's null, delayed, or its distribution shifts?
6. **a measurable improvement to the actual decision metric**, not just to a leaderboard number.

3.5 Everything above is stored as a “claim,” not a bare number

Each experiment doesn't just log a score — it logs structured claims with an evidence trail, explicit limitations, and (crucially) counter-evidence and “next discriminating test” fields (`DCLAB-R21`).

Example, verbatim from the registry: *“A cross-validation leader is not a universal algorithm winner and has not yet seen the locked holdout.”* That kind of self-qualifying statement is exactly the texture that makes a corpus useful for fine-tuning a model to reason carefully instead of overclaiming.

4. The 22 rules, as a plain-language checklist

The repo already distills its lessons into `knowledge/model_building_rules.jsonl` (22 rules with a statement, a reason, a failure signal, and a next test each). Read top-to-bottom, they *are* the “laws for future use cases” the brief asked for:

#	Stage	The law, in one line
1	Problem contract	Write the prediction moment, target window, action and error costs <i>before</i> looking at feature/target relationships.
2	Splitting	Split by the real independence unit and by time direction, before any preprocessing.
3	EDA	Fit anything target-aware — imputers, encoders, scalers, selectors — on the training fold only.
4	Leakage	Treat post-outcome, future, target-derived, cross-split, entity, and preprocessing leakage as separate failure modes, not one bucket.
5–6	Leakage	A suspicious name or a big safe-vs-unsafe score gap is a reason to <i>investigate</i> , not proof by itself.
7–9	Feature engineering	Start from a strong raw baseline; build features from real domain mechanisms first; every feature needs lineage, availability time, and a reproducible formula.
10	Feature reliability	Importance ranks a candidate; production-readiness needs availability + stability + monitoring on top.
11	Categoricals	Keep raw category meaning; fit encoders inside folds; never silently treat category codes as ordered numbers.
12	Missingness	Treat “missing” as an operational state to be measured and simulated, not just imputed away.
13–14	Model selection	No universal best algorithm — screen several families on identical folds; judge by mean performance <i>and</i> stability <i>and</i> runtime, not ROC-AUC alone.
15–16	Optimization	Tune only after data/leakage/features/metric/candidate are locked; require a predeclared, meaningful margin over a paired baseline.
17–18	Evaluation	Touch the final holdout exactly once; pick metrics and thresholds from the real business cost, not convention.
19	Generalization	Test temporal, geographic, and source-system slices whenever those shifts are plausible.
20	LLM governance	Let the LLM hypothesize and critique; keep splits, metrics, and artifacts under deterministic, auditable code.
21	Knowledge	Store every claim with its evidence, its limitations, and the next test that could break it.
22	Promotion	A model isn’t production-ready until provenance, independent confirmation, monitoring, fairness/privacy, and rollback are all reviewed — a good score is necessary, never sufficient.

5. Honest gap analysis — what's genuinely still missing

The request that triggered this review asked for three things that are **not yet fully covered** by what exists:

1. **Datasets literally sourced from Kaggle / HuggingFace, and more variety of problem type.** The current 11 datasets are all classic UCI tabular classification problems. There is no multiclass, no explicit time-series, no text-plus-tabular, and no heavily-imbalanced fraud-style dataset yet. Section 6 proposes ten concrete, freely downloadable candidates that specifically target this gap.
2. **A literal fine-tuning corpus.** `model_building_rules.jsonl`, `agent_memory.jsonl`, and `workflow_blocks.json` are excellent *retrieval* artifacts (they're built for RAG/agent memory), but they are not yet paired instruction→response examples in the shape an SFT job expects. Section 7 closes this gap with a ready-to-run script and a demonstrated, real output.
3. **A single narrative document meant for a first-time outside reader**, distinct from the (excellent but denser) `MODEL_BUILDING_FIELD_GUIDE.md`. This document is that piece.

One environment note worth being upfront about: **this chat session cannot reach kaggle.com or huggingface.co directly** (the sandboxed network here only allows a short list of package/registry domains). The existing `dclab_rnd` CLI and `campaign run` commands are exactly the right tool to run Round 2 with — they just need to run on your own machine or CI, where the repo's own `.venv` and network access already work. If you want an AI to drive that next round hands-on inside your actual repo and filesystem, **Claude Code** (desktop, CLI, or in your editor) is a better fit than this chat for that part, since it can execute against your real environment end-to-end.

6. Round 2: ten datasets chosen to stress *different* lessons

Each pick below is deliberately chosen to test a failure mode the first 50 experiments didn't cover, not just to add a row to a leaderboard.

Dataset	Where	Why it's a useful addition
Credit Card Fraud Detection	Kaggle (ULB)	Extreme class imbalance (~0.17% positive) — stress-tests metric choice (PR-AUC vs ROC-AUC) and resampling vs. cost-weighting.
IEEE-CIS Fraud Detection	Kaggle	Real production-style leakage traps (device/user IDs repeat across rows) — a genuine entity-leakage test case.
Home Credit Default Risk	Kaggle	Multiple linked tables — forces real feature-engineering-by-joining and point-in-time correctness across sources.
NASA Turbofan Degradation (C-MAPSS)	Kaggle / NASA	True time-series, remaining-useful-life regression — tests temporal splitting and window leakage directly.
HuggingFace tabular-benchmark (numerical/categorical suites)	Hugging Face Datasets	Curated, already-cleaned tabular benchmarks — good for isolating model-family comparison from data-cleaning noise.
Covertypes (Forest Cover Type)	UCI / Kaggle	Multiclass (7 classes), large (581k rows) — the campaign so far is entirely binary classification.
Telco/insurance-style multiclass churn or claims severity	Kaggle	Ordinal/multiclass target with real business cost asymmetry — tests custom cost-sensitive evaluation.
Give Me Some Credit	Kaggle	Smaller, noisier, more missingness than German Credit — a good stress test for the missingness-as-operational-state rule (DCLAB-R12).
Amazon Employee Access (or similar high-cardinality categorical set)	Kaggle	Very high-cardinality categoricals — tests encoding choices beyond what German Credit/Adult require.
A text+tabular hybrid (e.g., product return prediction with review text, or a HuggingFace tabular+text benchmark)	Hugging Face	Forces a decision about whether/how an embedding becomes a “feature,” and whether it passes the same leakage/availability tests as any other column.

Practical note: the existing `NEXT_EXPERIMENTS.md` already independently flagged three of the most valuable *next moves* — adding calibration/Brier-score benchmarks, measuring winner stability with repeated CV instead of point estimates, and replicating champions with full provenance. Doing those three **before** adding brand-new datasets will make Round 2's conclusions much more trustworthy, since right now only ~1.7% of historical runs carry full reproducibility provenance.

7. Turning this into an SLM fine-tuning corpus — done, with real output

A companion script, `build_sft_dataset.py`, converts the existing evidence files into a clean, chat-format instruction-tuning dataset:

- Each of the **22 rules** becomes 2 clean Q&A pairs (the rule + why it matters; and a “how would I notice I’m violating this” diagnostic framing).
- Each of the **10 workflow blocks** becomes 1 Q&A pair describing its step sequence.
- The **80 per-experiment claims** in `agent_memory.jsonl` are grouped by experiment and stage and turned into **50 evidence-grounded Q&A pairs**, with all provenance noise (git commit hashes, package versions, machine info, file paths) stripped out — keeping only the facts, risks, and limitations a model should actually learn to reproduce in its reasoning.
- Everything is deduplicated by content hash, shuffled with a fixed seed, and split 90/10 into `sft_train.jsonl` / `sft_val.jsonl`.

Running it just now against the real repo files produced:

```
Total examples : 104
Train / Val    : 94 / 10
By source:
  agent_memory           50
  model_building_rules   22
  model_building_rules_diagnostic 22
  workflow_blocks       10
```

That’s from only the small subset of files pulled in this session — the full local repo (579 historical records, `master_evidence_pack.json`’s ~2,400 lines, plus the full campaign) will yield several times more. Concretely, to grow this further:

1. **Run it against the full repo checkout**, not the excerpt used here — point `--claims` at `campaigns/model_building_50_v1/agent_memory.jsonl` and add a second pass over `knowledge/master_evidence_pack.json` (the same claim schema, at larger scale).
 2. **Add a “reasoning trace” variant per experiment**, following the repo’s own mandated LLM answer format (`AGENT_CONTEXT.md`): observed evidence → interpretation/uncertainty → decision → risks → next falsifiable test. This is a five-part chain-of-thought structure already defined in the repo — turning each experiment’s result JSON into one example in exactly that shape is the highest-value addition you can make next, because it teaches the *reasoning pattern*, not just facts.
 3. **Mix in negative/failed results** deliberately (the repo explicitly preserves them, `DCLAB-R21`) so the SLM learns “this didn’t work, here’s why” as often as “this worked.”
 4. **Keep a held-out slice of examples the model never trains on**, and periodically ask it the same questions a human reviewer would, to check it isn’t just memorizing phrasing.
 5. **Dilute with general instruction data** (10–30%) if you’re fine-tuning from a small base model — a corpus this specialized, on its own, risks narrowing the model’s general reasoning and writing ability.
-

8. Recommendations, in priority order

1. **Close the provenance gap before scaling up.** Re-run the current champions with the provenance-enabled runner and confirm reproducibility (`NEXT_EXPERIMENTS.md` , P1). A large fine-tuning corpus built on unverified numbers just teaches confident-sounding mistakes at scale.
2. **Add calibration and repeated-CV stability evidence** to the existing 11 datasets before adding new ones — cheap, and it directly strengthens every future training example that cites “the model with the best ROC-AUC.”
3. **Run Round 2 on the 10 datasets in Section 6**, locally or in CI (not in this chat), using the existing `dclab_rnd campaign run --dataset <name> machinery` — it’s already built for exactly this.
4. **Scale `build_sft_dataset.py`** against the full repo and add the five-part reasoning-trace format described in Section 7, item 2.
5. **Keep this document and `MODEL_BUILDING_FIELD_GUIDE.md` side by side** — this one is the front door for a new reader; the field guide is the reference once they’re oriented.
6. **Never let a good ROC-AUC skip the promotion checklist** (`DCLAB-R22`) — provenance, independent confirmation, monitoring, fairness/privacy, rollback. This is the rule most likely to get skipped under time pressure, and it’s the one the repo itself flags as non-negotiable.

Sources reviewed for this document: `README.md` , `knowledge/KNOWLEDGE_BASE.md` , `knowledge/NEXT_EXPERIMENTS.md` , `knowledge/DATA_QUALITY.md` , `knowledge/MODEL_BUILDING_FIELD_GUIDE.md` , `knowledge/model_building_rules.jsonl` , `knowledge/workflow_blocks.json` , `campaigns/model_building_50_v1/AGENT_CONTEXT.md` , `campaigns/model_building_50_v1/MODEL_BUILDING_WORKFLOW.md` , `campaigns/model_building_50_v1/CAMPAIGN_REPORT.md` , `campaigns/model_building_50_v1/agent_memory.jsonl` , `.cursor/rules/tabular-classification-playbook.mdc` , all pulled live from the public repository at the time of writing.